

Projektarbeit Pawsitters - A Pet Holiday Platform



von

Raphael Buller, Niklas Felix Ulbrich & Justus Krahel

Duale Hochschule Baden-Württemberg (DHBW) Heilbronn

Studiengang: Wirtschaftsinformatik – Software Engineering

Kurs: WI24A3

Dozent/-in: Dr. Ana-Maria-Christina Nicolaescu

Abgabedatum: 22.06.2026

Inhaltsverzeichnis

Abbildungsverzeichnis.....	4
Tabellenverzeichnis.....	5
Abkürzungsverzeichnis.....	6
1. Anforderungen und Architekturdokumentation.....	8
1.1 Anforderungsanalyse.....	8
1.1.1 Funktionale Anforderungen.....	8
1.1.2 Nicht-funktionale Anforderungen (ISO 25010).....	8
1.1.3 UseCases der Anwendung.....	9
1.2 Architekturbeschreibung.....	10
1.2.1 Schichtenmodell.....	10
1.2.2 Datenfluss am Beispiel einer Buchungsanfrage.....	10
1.2.3 API Schnittstellen.....	11
1.3 Datenmodell und Designentscheidungen.....	12
1.3.1 Domain-Modell (ER-Modell).....	12
1.3.2 Zentrale Designentscheidungen.....	13
1.3.3 Begründung der Datenbanktechnologie: Entscheidung für MySQL.....	14
1.4 Internationalisierung (i18n) und Design-System.....	14
2 Testdokumentation & Qualitätssicherung.....	14
2.1 Motivation: Warum wir testen.....	14
2.2 Testarten und technischer Einsatz.....	15
2.2.1 Unit-Tests (Backend & Frontend).....	15
2.2.2 Integrationstests (Backend).....	15
2.2.3 End-to-End Tests (Frontend).....	15
2.3 Quantitative Test-Übersicht.....	16
3 Security-Konzept.....	16
3.1 Sensible Daten im System.....	17
3.2 Bereits umgesetzte Sicherheitsmaßnahmen.....	17
3.2.1 Backend-Sicherheit (Spring Boot).....	17
3.2.2 Frontend-Sicherheit (Thymeleaf & Vue.js).....	18
3.3 Das Konzept „Shift Security Left“ und die Anwendung im Projekt.....	18
3.3.1 Definition des Konzepts.....	18
3.3.2 Praktische Anwendung bei Pawsitters.....	19
3.4 Roadmap: Zukünftige Maßnahmen und Production Hardening.....	19
3.4.1 Applikatorische Optimierungen (Kurzfristig).....	19
3.4.2 Infrastrukturelle Absicherung (Langfristig).....	20
4 Kollaboration und Entwicklungsprozess.....	21
4.1 Rollenverteilung und Verantwortlichkeiten.....	21
4.2 Vorgehensmodell und Methodik.....	21
4.2.1 Inkrementelle Entwicklung.....	22
4.2.2 Kanban als Steuerungsinstrument.....	22

4.2.3 Begründung der Methodik.....	23
4.3 Versionskontrolle und Branching-Strategie.....	23
4.3.1 Branch-Hierarchie und Workflow.....	23
4.3.2 Namenskonventionen.....	23
4.3.3 Quality Gates: Review-Prozess und Rulesets.....	24
4.3.4 Repository-Struktur.....	24
5.4 Continuous Integration & Continuous Delivery.....	24
5.4.1 Strategische Architektur-Entscheidungen im Deployment.....	26
6 Einsatz von Künstlicher Intelligenz.....	27
6.1 Status Quo im Entwicklungsprozess.....	27
Konkrete Anwendungsfälle und methodischer Mehrwert.....	27
Grenzen, Risiken und Herausforderungen.....	27
6.2 Future Build-In: KI-gestützte Marktplatz-Suche.....	27
7. Fazit.....	28
Anhang.....	30

Abbildungsverzeichnis

Abb. 1: Snapshot: Use-Case-Diagramm der Paw Sitters-Anwendung.....	9
Abb. 2: Teilhafte OpenAPI Spezifikation der Pawsitter Applikation.....	12
Abb. 3: Snapshot: Entity Relationship Modell der Pawsitters Anwendung.....	13
Abb. 4: Snapshot: Kanban Board vom Pawsittersprojekt.....	23
Abb. 5: Visualisierung der automatisierten CI/CD-Pipeline in GitHub Actions.....	25
Abb. 6: Beispiel der KI-Unterstützten Suche am beispiel von Mobile.de.....	28

Tabellenverzeichnis

Tabelle 1: Auflistung der Anzahl an Backend Tests und deren Art, Technologie und Fokus	15
--	----

Tabelle 2: Auflistung der Anzahl an Frontend Tests und deren Art, Technologie und Fokus	15
---	----

Abkürzungsverzeichnis

MVP	engl. Minimal Viable Product (Minimal überlebensfähiges Produkt)
UC	engl. Use Case (Anwendungsfall)
UML	engl. Unified Modeling Language (Vereinheitlichte Modellierungssprache)
ERM	engl. Entity-Relationship-Modell (Gegenstand-Beziehung-Modell)
ORM	engl. Object-Relational Mapping (Objekt-relationales Mapping)
RDBMS	engl. Relational Database Management System
ACID	engl. Atomicity, Consistency, Isolation, Durability
MVC	engl. Model-View-Controller (Modell-Ansicht-Steuerung)
UI	engl. User Interface (Benutzeroberfläche)
UX	engl. User Experience (Nutzererlebnis)
DTO	engl. Data Transfer Objects (Datentransferobjekte)
PR	engl. Pull Request (Änderungsanforderung)
SDLC	engl. Software Development Life Cycle (Software-Entwicklungslebenszyklus)
API	engl. Application Programming Interface (Programmierschnittstelle)
JSON	engl. JavaScript Object Notation (Standardisiertes Datenaustauschformat)
QS	Qualitätssicherung
E2E	engl. End-to-End (Ende-zu-Ende-Test)
CI	engl. Continuous Integration (Kontinuierliche Integration)
CD	engl. Continuous Delivery / Continuous Deployment
WIP-Limit	engl. Work in Progress Limit
SCM	engl. Source Code Management
GHCR	engl. GitHub Container Registry
DSGVO	Datenschutz-Grundverordnung
PII	engl. Personally Identifiable Information (Personenbezogene Daten)

JWT engl. JSON Web Token

XSS engl. Cross-Site Scripting

CSRF engl. Cross-Site Request Forgery

RBAC engl. Role-Based Access Control (Rollenbasierte Zugriffskontrolle)

NFCengl. Normalization Form Canonical Composition (Unicode-Normalisierungsform)

SHA-256 engl. Secure Hash Algorithm 256-Bit

MIME-Type engl. Multipurpose Internet Mail Extensions Type

SCA engl. Software Composition Analysis

CVE engl. Common Vulnerabilities and Exposures

SAST engl. Static Application Security Testing

WAF engl. Web Application Firewall

DDoS engl. Distributed Denial of Service (Verteilter Überlastungsangriff)

VPC engl. Virtual Private Cloud (Isoliertes, privates Cloud-Netzwerk)

TLS engl. Transport Layer Security

NIST engl. National Institute of Standards and Technology

LLM engl. Large Language Model (Großes Sprachmodell / KI-Typ)

DHBW Duale Hochschule Baden-Württemberg

WI24A3 Kursbezeichnung Wirtschaftsinformatik Jahrgang 2024

1. Anforderungen und Architekturdokumentation

1.1 Anforderungsanalyse

Die Anforderungsanalyse bildet die Grundlage für die technische Umsetzung von **Pawsitters**. Sie teilt sich gemäß den Vorlesungsstandards in funktionale und nicht-funktionale Anforderungen auf.

1.1.2 Funktionale Anforderungen

Das System erfüllt die im Projektportfolio definierten Kernfunktionen, die durch spezifische User Stories abgebildet werden:

- **Profil- und Rollenmanagement:** Nutzer können ein universelles Profil erstellen und flexibel zwischen den Rollen „Tierhalter“ (Owner) und „Gastgeber“ (Host) wechseln.
- **Haustier-Registrierung:** Tierhalter können ihre Haustiere mit relevanten Merkmalen (Rasse, Alter, Bedürfnisse) im System hinterlegen.
- **Marktplatz und Vermittlung:** Gastgeber definieren Verfügbarkeiten und Preise. Tierhalter können über eine Such- und Filterlogik passende Angebote finden.
- **Interaktions-Workflow:** Über ein integriertes Chat-System können Details geklärt werden, bevor eine verbindliche Buchungsanfrage gestellt, angenommen oder abgelehnt wird.

1.1.2 Nicht-funktionale Anforderungen (ISO 25010)

Zur Sicherung der Softwarequalität wurden drei zentrale Merkmale nach ISO 25010 priorisiert:

1. **Wartbarkeit (Maintainability):** Durch eine strikte Schichtenarchitektur und die Einhaltung des **Single Responsibility Prinzips** ist der Code modular aufgebaut. Änderungen an der Datenbanklogik beeinflussen nicht direkt die Benutzeroberfläche.
2. **Benutzbarkeit (Usability):** Ein einheitliches Corporate Design mittels Tailwind CSS und reaktive Komponenten durch Vue.js sorgen für eine intuitive Bedienung. Die Internationalisierung (i18n) ermöglicht die Nutzung in verschiedenen Sprachen, und eine einfach zu verstehende UI wie auch einheitliche APIs machen es nochmal einfacher zu benutzen.
3. **Sicherheit (Security):** Gemäß dem Prinzip „Shift Security Left“ werden Sicherheitsaspekte wie Passwort-Hashing und die Trennung von sensiblen Daten durch DTOs bereits im Design berücksichtigt.

4. **Nachvollziehbarkeit (Traceability):** Die vollständige Historie der Code-Änderungen wird durch **Git** verwaltet. Alle externen Schnittstellen (APIs) werden mittels **OpenAPI** dokumentiert, um Klarheit über den API-Vertrag zu gewährleisten.

1.1.3 UseCases der Anwendung

Für die Pawsitters Applikation wurden bestimmte Use Cases bestimmt, die für die Funktion und Einlösung der funktionalen Anforderungen unerlässlich sind. Diese sind in der untenstehenden Grafik veranschaulicht:

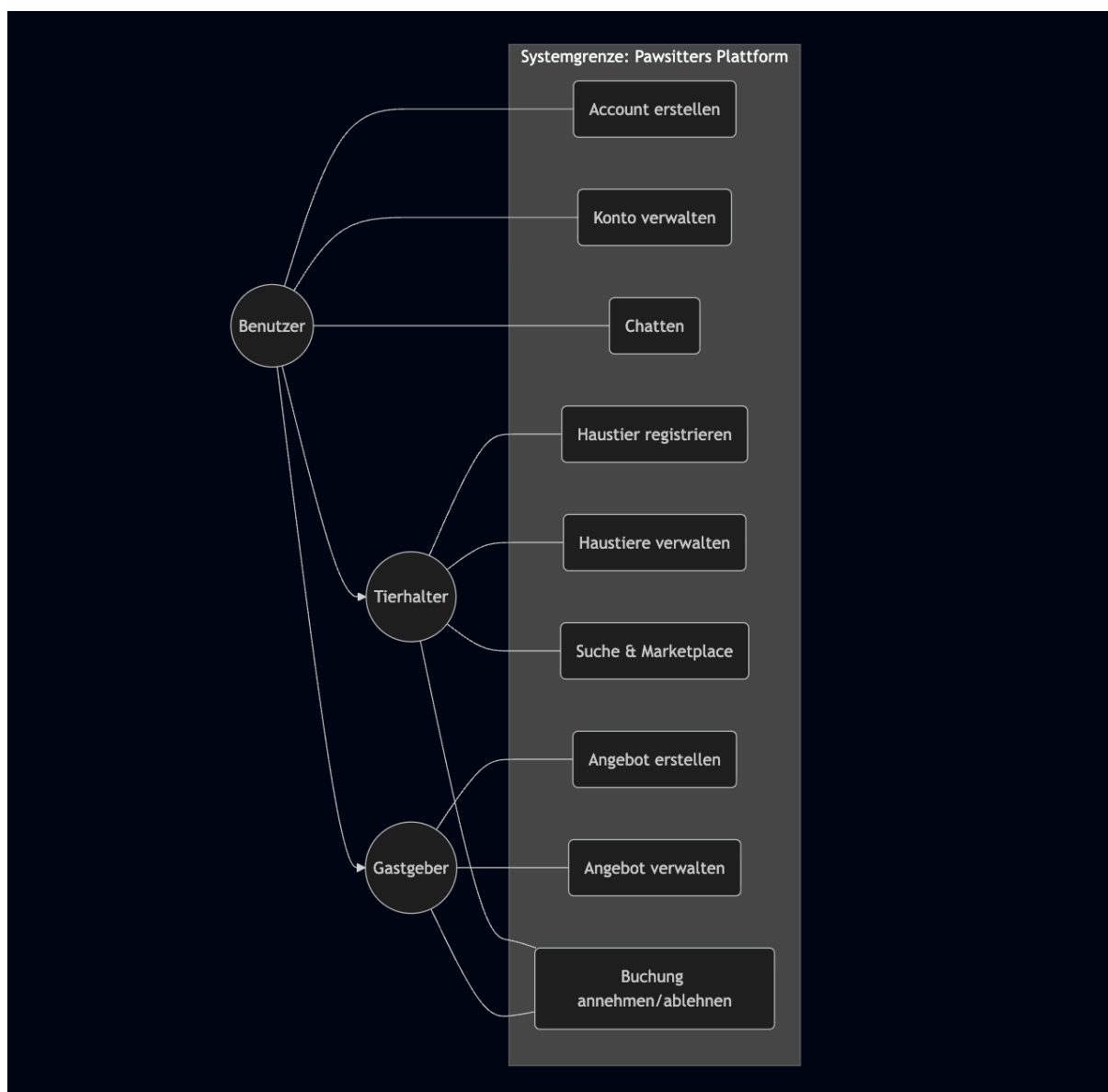


Abb. 1: Snapshot: Use-Case-Diagramm der Paw Sitters-Anwendung

Bitte beachten Sie: Die vollständigen Use Case-Beschreibungen finden Sie im **Anhang 1 (Use Case Beschreibungen der Pawsitters Applikation)**.

1.2 Architekturbeschreibung

Die Architektur von „Pawsitters“ ist als klassische Schichtenarchitektur (Layered Monolith) konzipiert, die das Model-View-Controller (MVC) Muster um eine dedizierte Dienstschicht (Service Layer) erweitert. Diese Wahl stützt sich auf die im Rahmen der Vorlesung behandelten Prinzipien zur Komplexitätsbeherrschung und Modularisierung. Durch die konsequente Separation of Concerns (Trennung der Belange) wird sichergestellt, dass Geschäftslogik, Datenhaltung und Präsentation voneinander entkoppelt sind. Dies maximiert die Wartbarkeit und Testbarkeit des Systems gemäß ISO 25010. Während komplexere Ansätze wie Microservices für den Umfang des Projekts einen unverhältnismäßigen infrastrukturellen Overhead bedeutet hätten, bietet dieser Schichtenaufbau die optimale Balance zwischen technischer Struktur und effizienter Entwicklungsgeschwindigkeit innerhalb der Gruppe.

1.2.1 Schichtenmodell

Das System ist in vier logische Ebenen unterteilt:

- **Presentation Layer (Frontend):** Ein hybrider Ansatz aus Thymeleaf (für das serverseitige Rendering der Grundstruktur) und Vue.js (für dynamische Inhalte wie Chats). Tailwind CSS gewährleistet ein konsistentes Styling.
- **API / Controller Layer (Controller):** REST-Controller nehmen Anfragen entgegen, validieren die Eingaben und delegieren die Verarbeitung an den Service Layer. Hier findet die Entkopplung durch DTOs (Data Transfer Objects) statt.
- **Service Layer (Service):** Enthält die zentrale Geschäftslogik (z. B. Validierung von Buchungszeiträumen). Diese Schicht koordiniert die Transaktionen und stellt sicher, dass die Datenintegrität gewahrt bleibt.
- **Persistence Layer (Repository):** Nutzt Spring Data JPA und Hibernate, um das Domain-Modell auf eine relationale Datenbank (H2 für Entwicklung/Testing) abzubilden.

1.2.2 Datenfluss am Beispiel einer Buchungsanfrage

1. **Frontend:** Der Nutzer klickt auf „Anfrage senden“. Eine Vue-Komponente sendet ein JSON-Objekt an `POST /api/booking-requests`.
2. **Controller:** Nimmt das `BookingRequestDTO` entgegen und prüft die syntaktische Korrektheit.
3. **Service:** Der `BookingService` prüft fachlich, ob der Gastgeber im gewählten Zeitraum verfügbar ist.
4. **Repository:** Die Anfrage wird als Entität in der Datenbank persistiert.

1.2.3 API Schnittstellen

Die Kommunikation zwischen der Benutzeroberfläche (Presentation Layer) und dem Server (Backend) erfolgt über eine REST-konforme API-Schnittstelle. Diese Schnittstellen sind im Controller Layer angesiedelt und bilden das Bindeglied zwischen den Client-Anfragen und der internen Geschäftslogik. Jede Anfrage wird hierbei als HTTP-Request (GET, POST, PUT, DELETE) entgegengenommen, validiert und anschließend an den Service Layer delegiert. Ein zentrales Entwurfsprinzip bei der Gestaltung der Schnittstellen ist die Verwendung von Data Transfer Objects (DTOs). Anstatt die internen Datenbank-Entities direkt nach außen zu exponieren, werden spezifische Objekte für den Datenaustausch definiert. Dies gewährleistet eine lose Kopplung: Änderungen am Datenmodell der Persistenzschicht führen nicht zwangsläufig zu einem „Breaking Change“ in der API. Zudem wird so sichergestellt, dass nur die für den jeweiligen Anwendungsfall relevanten Informationen übertragen werden. Der Datenaustausch erfolgt dabei standardisiert im JSON-Format, was eine hohe Interoperabilität zwischen den Systemkomponenten ermöglicht, und für das debuggen auch besser lesbar ist.

Um die Klarheit und Nutzbarkeit der implementierten REST-Schnittstellen zu maximieren, wurde die gesamte API nach dem **OpenAPI-Standard (vormals Swagger)** dokumentiert. Die vollständige Spezifikation ist in der zentralen Datei

dhbw_pawsitters_se2-openapi.yaml hinterlegt. Diese standardisierte Dokumentation gewährleistet, dass sowohl das Frontend-Team als auch zukünftige Entwickler den API-Vertrag präzise verstehen und automatisiert Client-Code generieren können.

Pawsitters API 1.1.0 OAS 3.0	
OpenAPI-Dokumentation fuer die Pawsitters JSON-API mit einheitlichem Response-Envelope.	
Servers http://localhost:8080	
Auth Endpunkte fuer Login, Logout, Registrierung und Sessionstatus.	
POST	/api/auth/register Registrieren und einloggen
POST	/api/auth/login Einloggen
POST	/api/auth/logout Ausloggen
GET	/api/auth/session Sessionstatus abrufen
Users Endpunkte zur Benutzerverwaltung.	
Pets Endpunkte zur Haustierverwaltung.	
GET	/api/pets Eigene Haustiere abrufen
POST	/api/pets Haustier erstellen
PUT	/api/pets/{id} Haustier aktualisieren
DELETE	/api/pets/{id} Haustier loeschen
POST	/api/pets/{id}/image Haustierbild hochladen
Offers Endpunkte zum Erstellen und Verwalten von Betreuungspaketen.	
POST	/api/offers Betreuungspaket als Entwurf erstellen
GET	/api/offers/{id} Eigenes Betreuungspaket abrufen
PATCH	/api/offers/{id}/publish Betreuungspaket veroeffentlichen
PATCH	/api/offers/{id}/withdraw Betreuungspaket zurueckziehen
Marketplace Endpunkte zur Gastgeber- und Angebotsuebersicht, Suche und Filteroptionen.	
GET	/api/marketplace/hosts Gastgeber abrufen
GET	/api/marketplace/offers Veroeffentlichte Betreuungspakete abrufen
GET	/api/marketplace/hosts/search Gastgeber nach Tierart und Postleitzahl suchen
GET	/api/marketplace/filters Veruegbare Marketplace-Filter abrufen

Abb. 2: Teilhafte OpenAPI Spezifikation der Pawsitter Applikation

Anmerkung: Eine vollständige tabellarische Auflistung aller implementierten API-Schnittstellen findet sich im Anhang 2: API Schnittstellen der Paw Sitters Applikation.

1.3 Datenmodell und Designentscheidungen

1.3.1 Domain-Modell (ER-Modell)

Das Datenmodell ist so strukturiert, dass es die fachliche Komplexität der Plattform widerspiegelt. Die zentrale Entität ist der **User**, der über ein **HostProfile** verfügen kann.

Haustiere (**Pet**) sind dem User direkt zugeordnet. Ein **BookingRequest** verknüpft einen Tierhalter, ein Tier und ein Gastgeberprofil.

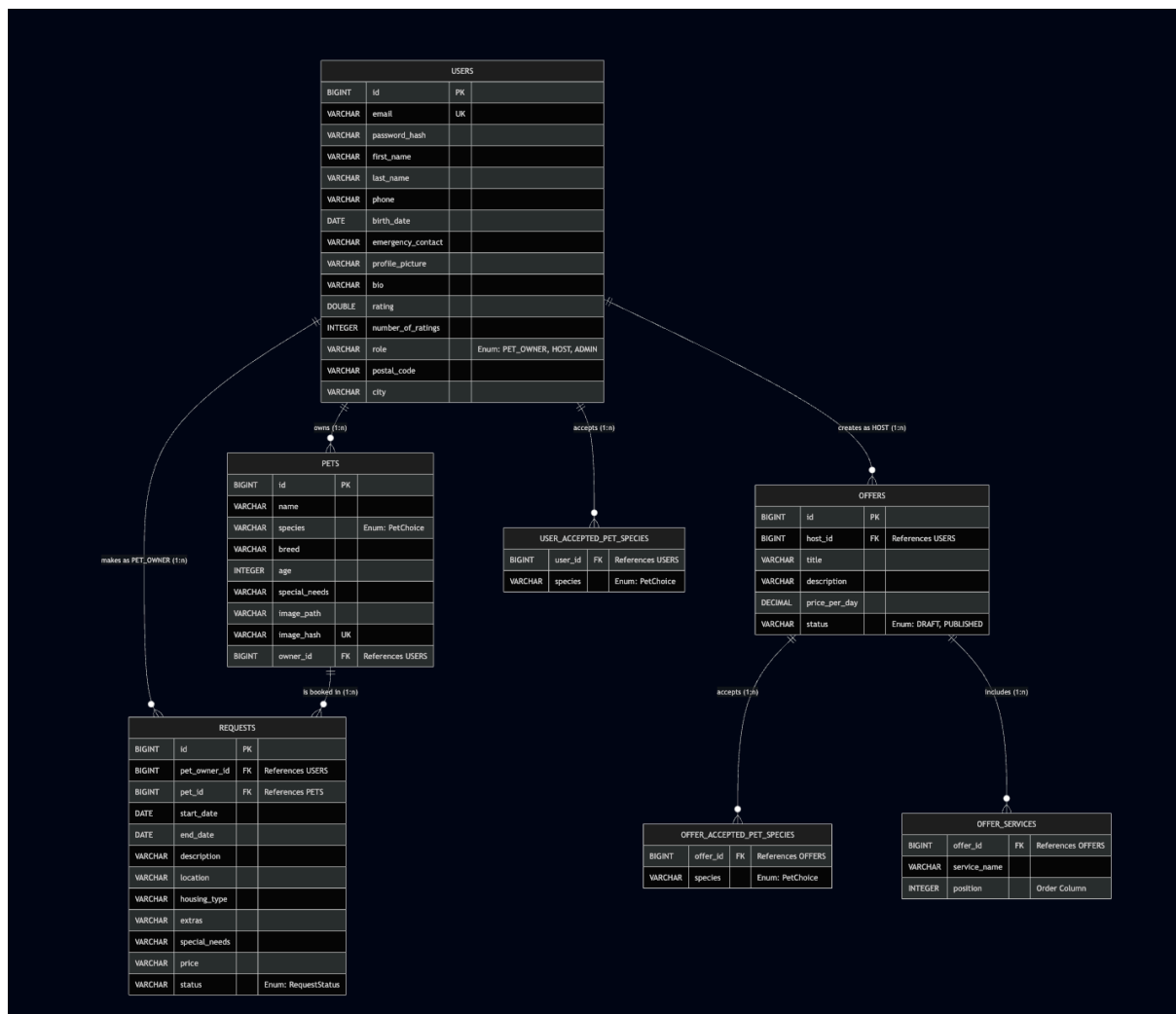


Abb. 3: Snapshot: Entity Relationship Modell der Pawsitters Anwendung

1.3.2 Zentrale Designentscheidungen

- **DTO-Pattern:** Um „Overexposure“ zu vermeiden (Sicherheitsrisiko, bei dem interne Datenbankfelder an das Frontend gelangen), werden für alle API-Schnittstellen DTOs verwendet. Dies ermöglicht eine saubere Trennung zwischen API-Vertrag und internem Datenmodell.
- **Separation of Concerns:** Durch die strikte Trennung von Controller (Infrastruktur), Service (Logik) und Repository (Daten) bleibt das System testbar und erweiterbar.
- **Dependency Injection:** Nutzt Spring Data JPA und Hibernate, um das Domain-Modell auf eine relationale Datenbank (MySQL für Entwicklung/Produktion) abzubilden.

1.3.3 Begründung der Datenbanktechnologie: Entscheidung für MySQL

Für den produktiven Einsatz und die lokale Entwicklung (außerhalb isolierter Tests) wurde MySQL als relationales Datenbankmanagementsystem (RDBMS) gewählt. Diese Entscheidung basiert auf folgenden architektonischen und funktionalen Überlegungen:

- **Datenintegrität und relationales Modell:** Das komplexe, stark vernetzte Domänenmodell (Nutzer, Haustiere, Buchungsanfragen) erfordert die durch MySQL gesicherte Datenintegrität mithilfe von Fremdschlüsseln und Constraints.
- **Transaktionssicherheit (ACID):** Die strikte Einhaltung der ACID-Eigenschaften ist für das Management von Buchungs- und Stornierungsprozessen zwingend erforderlich, um Konsistenz bei gleichzeitigen Schreibzugriffen zu garantieren.
- **Nahtlose Spring-Integration:** MySQL bietet optimale Unterstützung in der Java-Welt. Das Zusammenspiel mit Spring Data JPA und Hibernate (ORM) über den MySQLDialect ermöglicht die effiziente Übersetzung von Features wie Paginierung und komplexen Joins.
- **Produktionsreife und Performance:** Im Gegensatz zu H2 (nur für isolierte Tests geeignet) bietet MySQL die notwendige Robustheit, skalierbare Performance, Connection-Pooling (HikariCP) und etablierte Backup-Strategien für den produktiven Betrieb und hohe Nebenläufigkeit.
- **Entwickler-Ergonomie und Parität:** Die Bereitstellung via Docker stellt eine identische, standardisierte Entwicklungsumgebung sicher, was die Parität zwischen Entwicklungs- und Produktionsumgebung gewährleistet.

1.4 Internationalisierung (i18n) und Design-System

Die Architektur unterstützt Mehrsprachigkeit durch eine zentrale Ressourcenverwaltung. Alle statischen Texte sind aus dem Code extrahiert und in `messages.properties` (Backend/Thymeleaf) sowie JSON-Dateien (Frontend/Vue.js) hinterlegt. Das Corporate Design wird durch eine zentrale `tailwind.config.js` gesteuert, welche die Markenidentität (Farben, Abstände, Typografie) systemweit vorgibt.

2 Testdokumentation & Qualitätssicherung

Die kontinuierliche Qualitätssicherung bildet einen Kernpfeiler des Entwicklungsprozesses der Plattform „Pawsitters“. Um die Stabilität, Sicherheit und Benutzbarkeit des Gesamtsystems gemäß den Qualitätsmerkmalen der ISO 25010 zu garantieren, wurde eine umfassende, automatisierte Teststrategie implementiert.

2.1 Motivation: Warum wir testen

Der Verzicht auf ausschließlich manuelle Prüfprozesse und der Fokus auf automatisierte Softwaretests stützt sich auf drei strategische Zielsetzungen:

1. **Vermeidung von Regressionen (Stabilität):** Bei funktionalen Erweiterungen oder Refactorings wird sichergestellt, dass bestehende Kernfeatures (wie Registrierung, Marktplatz-Filter oder der Echtzeit-Chat) unberührt bleiben und fehlerfrei weiterlaufen.
2. **Absicherung der Security („Shift Security Left“):** Sicherheitskritische Komponenten wie die JWT-Validierung, rollenbasierte Zugriffskontrollen (RBAC) sowie Schutzmechanismen gegen Brute-Force-Angriffe (Rate Limiting) werden fortlaufend überprüft.
3. **Beherrschung von Edge Cases (Grenzfällen):** Das System wird gezielt mit invaliden Zuständen konfrontiert (z. B. fehlerhaftes JSON, unzulässige Buchungszeiträume, übergroße Datei-Uploads), um ein robustes Verhalten und eine kontrollierte Fehlerbehandlung zu verifizieren.

2.2 Testarten und technischer Einsatz

Das System differenziert technologisch und methodisch zwischen drei verschiedenen Testausprägungen, um jede Schicht der Anwendungsarchitektur optimal abzudecken:

2.2.1 Unit-Tests (Backend & Frontend)

- **Warum und wofür:** Unit-Tests prüfen die kleinste isolierbare Einheit des Quellcodes, meist einzelne Methoden oder isolierte Service-Klassen, vollständig entkoppelt von äußeren Einflüssen wie Datenbanken oder Netzwerk-Schnittstellen.
- **Einsatz:** Im Backend werden Services mithilfe von **JUnit** und **Mockito** getestet, indem Abhängigkeiten (Repositories) simuliert (gemockt) werden. Im Frontend dienen Unit-Tests (ausgeführt über den Node Test Runner) dazu, komplexe Daten-Transformationen wie das Parsing von Git-Graphen oder OpenAPI-Spezifikationen mathematisch exakt abzusichern.

2.2.2 Integrationstests (Backend)

- **Warum und wofür:** Integrationstests überprüfen das korrekte Zusammenspiel mehrerer Komponenten und Architekturschichten über deren Grenzen hinweg.
- **Einsatz:** Mithilfe des Spring-Boot-Testcontexts und `@AutoConfigureMockMvc` werden vollständige HTTP-Zyklen simuliert. Hierbei wird getestet, ob die REST-Controller Anfragen korrekt validieren, DTOs fehlerfrei mappen, die Transaktionssicherheit im Service-Layer gewahrt bleibt und die Datenhaltung im Persistence-Layer wie erwartet reagiert.

2.2.3 End-to-End Tests (Frontend)

- **Warum und wofür:** E2E-Tests simulieren das reale Verhalten eines Endnutzers, indem sie die Anwendung in einem echten, ferngesteuerten Browser aufrufen, Klicks tätigen und Formulare ausfüllen.
- **Einsatz:** Das Frontend nutzt das Framework **Playwright**. Es testet komplexe, schichtenübergreifende Benutzerinteraktionen, wie den vollständigen

Registrierungs-Flow, reaktive Sprachumschaltungen (i18n) im Login-Modal sowie das responsive Verhalten und visuelle Begrenzungen (Z-Index-Stapelung) von UI-Komponenten.

2.3 Quantitative Test-Übersicht

Das Projekt verfügt über einen aktiven Bestand von insgesamt **165** automatisierten Tests. Die nachfolgenden Tabellen schlüsseln die Verteilung der Testdatenbestände getrennt nach Systemarchitektur auf.

Test-Art	Test-Framework / Technologie	Fokus / Ebene	Anzahl Tests
Unit-Tests	JUnit 5 & Mockito	Isolierte Überprüfung der Core-Services und Domänenlogik	29
Integrationstests	Spring Boot Test & MockMvc	Schichtenübergreifende Validierung von APIs, Security und DB-Persistenz	102
Gesamt (Backend)			131

Tabelle 1: Auflistung der Anzahl an Backend Tests und deren Art, Technologie und Fokus

Test-Art	Test-Framework / Technologie	Fokus / Ebene	Anzahl Tests
Unit-Tests	Node Test Runner	Mathematische Daten-Transformationen & OpenAPI-Parsing	15
End-to-End (E2E)	Playwright	Browserübergreifende Nutzerflüsse, i18n & UI-Interaktionen	19
Gesamt (Frontend)			34

Tabelle 2: Auflistung der Anzahl an Frontend Tests und deren Art, Technologie und Fokus

Anmerkung: Die Gesamte Testdokumentation wobei jeder Test genau erklärt ist, finden sie auch im Repository unter der Datei `TEST_DOCUMENTATION.md` zu finden.

3 Security-Konzept

Die Gewährleistung von Informationssicherheit und Datenschutz besitzt bei der Plattform „Pawsitters“ höchste Priorität. Entsprechend dem modernen Software-Engineering-Paradigma wurde Sicherheit nicht als abschließende Phase, sondern

als integraler Bestandteil des gesamten Entwicklungszyklus betrachtet wie in den Anforderungen des Projektes im Rahmen der Vorlesung gefordert. Dieses Kapitel analysiert die schutzbedürftigen Daten, dokumentiert die bereits implementierten sowie zukünftig notwendigen Sicherheitsmechanismen und beschreibt die Anwendung des Prinzips „Shift Security Left“.

Anmerkung: Dieses Konzept ist auch im Repository unter der Datei `SECURITY_KONZEPT.md` zu finden.

3.1 Sensible Daten im System

Innerhalb der Plattform werden verschiedene Kategorien von Daten verarbeitet, die gemäß der Datenschutz-Grundverordnung (DSGVO) sowie allgemeiner IT-Sicherheitsstandards eines besonderen Schutzes bedürfen:

- **Personenbezogene Daten (PII):** Klarnamen, E-Mail-Adressen, Telefonnummern, Geburtsdaten und Notfallkontakte der Tierhalter und Gastgeber.
- **Authentifizierungsdaten:** Kryptografisch gehashte Passwörter, kryptografisch signierte JSON Web Tokens (JWT) und Session-Cookies.
- **Private Kommunikation:** Vertrauliche Chat-Nachrichten zwischen den Vertragsparteien sowie die innerhalb des Chats geteilten privaten Medien-Anhänge.
- **Buchungs- und Transaktionsdaten:** Indirekte Finanzdaten wie vereinbarte Wochenpreise, Angebote, Buchungszeiträume und kalendarische Verfügbarkeiten.
- **Nutzergenerierte Medien:** Profilbilder der Akteure, Bildmaterial der Haustiere sowie visuelle Repräsentationen der Betreuungsangebote.

3.2 Bereits umgesetzte Sicherheitsmaßnahmen

Die Applikation verfügt bereits in der aktuellen Entwicklungsphase über eine robuste Sicherheitsarchitektur, die sowohl serverseitig als auch clientseitig greift.

3.2.1 Backend-Sicherheit (Spring Boot)

- **Zustandlose JWT-Authentifizierung:** Die Authentifizierung erfolgt zustandlos über signierte JSON Web Tokens (JWT). Beim Logout wird das entsprechende Token auf eine serverseitige Sperrliste (*Revocation List*) gesetzt, um eine sofortige Ungültigkeit zu erzwingen.
- **Kryptografisches Hashing:** Passwörter werden unter Verwendung des industriebewährten `BCrypt`-Verfahrens gehashed, nachdem zuvor eine Unicode-Normalisierung (`NFC`) der Eingabe stattgefunden hat.
- **Strikte Passwort-Policy:** Über die serverseitige *Bean Validation* wird eine Mindestlänge von 15 Zeichen erzwungen. Schwache Passwörter, gängige Sequenzen sowie die Nutzung von eigenen Namens- oder E-Mail-Bestandteilen werden algorithmisch blockiert.
- **Brute-Force-Schutz (Rate Limiting):** Zum Schutz vor automatisierten Anmeldeversuchen sperrt ein integrierter Mechanismus IP-Adressen und

Benutzerkonten nach fünf fehlgeschlagenen Logins für einen Zeitraum von 15 Minuten.

- **Rollenbasierte und objektbezogene Autorisierung (RBAC):** Neben der klassischen Trennung zwischen Benutzer- und Administratorrechten verhindern feingranulare, objektbezogene Prüfungen (*Method Security*), dass unbefugte Dritte fremde Daten (Tiere, Angebote, Chats) modifizieren oder WebSocket-Topics abhören können.
- **Upload-Validierung:** Bilder werden strikt auf ihren MIME-Type (`image/...`) geprüft. Dateinamen werden serverseitig neu generiert, die Dateigröße ist auf 5 MB limitiert und eine SHA-256-Prüfsumme verhindert Redundanzen. Eine vorgeschaltete Bilddekodierung fängt bössartige Payload (z. B. in `.jpg` versteckte PHP-Shells) ab.
- **Data Minimization:** Durch den konsequenten Einsatz von Data Transfer Objects (DTOs) wird sichergestellt, dass interne Entitätsfelder wie Passwort-Hashes niemals die Systemgrenze in Richtung Client passieren. Das globale Error-Handling fängt zudem Stacktraces ab, um das Ausspähen von Systeminterna (*Information Leakage*) zu unterbinden.

3.2.2 Frontend-Sicherheit (Thymeleaf & Vue.js)

- **XSS-Prävention:** Durch die ausschließliche Verwendung von `th:text` (Thymeleaf) und `v-text` (Vue.js) werden sämtliche Benutzereingaben bei der Ausgabe im Browser automatisch maskiert. Die potenziell unsichere Interpretation von rohem HTML ist im Quellcode vollständig unterbunden.
- **Sicheres Cookie-Handling:** Die Speicherung und Übertragung des Authentifizierungstokens erfolgt über `HttpOnly`-Cookies, wodurch ein Auslesen des Tokens via JavaScript (z. B. durch Cross-Site-Scripting) technisch unmöglich ist.
- **Infrastruktureller Schutz:** Externe Hyperlinks sind durch das Attribut `rel="noopener noreferrer"` gegen *Tabnabbing*-Angriffe geschützt. Clientseitige *Route Guards* sichern zudem die Benutzeroberfläche ab, während das Backend die primäre Autorisierungsprüfung durchführt.

3.3 Das Konzept „Shift Security Left“ und die Anwendung im Projekt

3.3.1 Definition des Konzepts

Das Prinzip „Shift Security Left“ beschreibt das Paradigma, Sicherheitsaspekte und automatisierte Prüfverfahren nicht erst am Ende des Software-Entwicklungslebenszyklus anzusiedeln, sondern so weit wie möglich nach „links“, also in die frühen Phasen der Planung, des Designs und der kontinuierlichen Integration (CI/CD), zu verschieben. Dadurch werden Schwachstellen frühzeitig durch die Entwickler selbst erkannt und können kostengünstig behoben werden, bevor der Code in eine produktive Umgebung gelangt.

3.3.2 Praktische Anwendung bei Pawsitters

Im Rahmen des Pawsitters-Projekts wird dieser Ansatz durch folgende technische und prozessuale Maßnahmen operationalisiert:

1. **Software Composition Analysis (SCA):** Über automatisierte Tools (wie *Dependabot*) wird das Projekt kontinuierlich auf bekannte Sicherheitslücken (CVEs) in Drittanbieter-Bibliotheken (Spring Boot Starter, npm-Pakete) untersucht und warnt das Team proaktiv.
2. **Automated Security Testing:** Sicherheitsanforderungen wurden direkt in funktionalen Integrationstests abgebildet. So validiert der *AuthIntegrationTest* das Rate-Limiting und Autorisierungsfehler automatisiert bei jedem Build.
3. **Security by Design:** Bereits bei der Erstellung von User Stories im Kanban-Board werden Sicherheitskriterien als feste Akzeptanzkriterien definiert (z. B. „Zugriff auf Chat-Anhänge nur für verifizierte Chat-Teilnehmer“).

3.4 Roadmap: Zukünftige Maßnahmen und Production Hardening

Um die Anwendung aus dem aktuellen Prototypenstatus in ein reales, hochsicheres Marktprodukt zu überführen, müssen im Rahmen des *Production Hardening* folgende nachgelagerte Maßnahmen implementiert werden:

3.4.1 Applikatorische Optimierungen (Kurzfristig)

1. **Datenschutz-Reduktion (Public DTOs):** Öffentliche Endpunkte wie */api/users/{id}* müssen für anonyme Gäste stark reduzierte Datenstrukturen ausgeben. Sensible Daten wie Telefonnummern oder Notfallkontakte dürfen erst nach einer erfolgreichen Autorisierungsprüfung (z. B. bei einer aktiven Buchung) freigegeben werden.
2. **Autorisierter Medien-Zugriff:** Der direkte, statische Zugriff auf Chat-Anhänge im Verzeichnis */uploads/messages/* muss unterbunden werden. Die Auslieferung von Anhängen hat zwingend über einen authentifizierten Controller zu erfolgen, der die Chat-Zugehörigkeit validiert.
3. **Umfassender CSRF-Schutz:** Aufgrund der Nutzung von Auth-Cookies muss zur Absicherung statusverändernder Operationen (POST, PUT, DELETE) ein CSRF-Token-Verfahren implementiert oder eine strikte *SameSite=Strict*-Richtlinie für Cookies erzwungen werden.
4. **Secret Scanning:** Der optionale Einsatz von Tools wie *Gitleaks* als Pre-Commit-Hook stellt sicher, dass kryptografische Schlüssel, Passwörter oder das *JWT_SECRET* niemals versehentlich in die Versionskontrolle gepusht werden.

5. **Token-Isolierung:** Das JWT darf beim Login-Prozess ausschließlich im **HttpOnly**-Cookie und nicht zusätzlich im JSON-Response-Body übertragen werden, um jeglichen Angriffsvektor über den Client-Speicher zu eliminieren.
6. **Statische Code-Analyse (SAST) in der Pipeline:** Die Integration von statischen Code-Analyse-Werkzeugen in die GitHub-Actions-Pipeline ermöglicht es, jeden Pull Request automatisiert auf Sicherheitsrisiken wie SQL-Injections oder Hardcoded Secrets zu scannen, bevor ein Merge erlaubt wird.

3.4.2 Infrastrukturelle Absicherung (Langfristig)

- **Web Application Firewall (WAF):** Vorschaltung einer WAF (z. B. Cloudflare, AWS WAF) zur proaktiven Filterung von böartigem Traffic, SQL-Injections, Cross-Site-Scripting und zur Abwehr von Distributed-Denial-of-Service-Angriffen (DDoS).
- **Netzwerksegmentierung:** Platzierung des Backend-Applikationsservers und des relationalen Datenbanksystems (PostgreSQL) in einem privaten, vom Internet isolierten Subnetz (VPC). Der Zugriff wird ausschließlich über ein vorgelagertes API-Gateway oder einen Load Balancer erlaubt.
- **Zentralisiertes Secrets-Management:** Die Verwaltung von produktiven Datenbank-Passwörtern und kryptografischen Schlüsseln muss von Umgebungsvariablen in einen dedizierten, verschlüsselten Tresor (z. B. *HashiCorp Vault* oder *AWS Secrets Manager*) verlagert werden.
- **Verschlüsselung auf allen Ebenen:** Konsequente Erzwingung von TLS 1.3 für Daten in Bewegung (*Data in Transit*) sowie vollständige Verschlüsselung der physischen Datenbank-Festplatten (*Data at Rest*).
- **Audits:** Durchführung regelmäßiger, professioneller Penetrationstests durch externe IT-Sicherheitsdienstleister vor Major-Releases.

4 Kollaboration und Entwicklungsprozess

Der Erfolg des Projekts „Pawsitters“ basierte auf einer agilen Arbeitsweise und einer klaren, aber flexiblen Rollenverteilung. Um die Komplexität der Anforderungen zu beherrschen, wurde ein iteratives Vorgehen gewählt, das eine kontinuierliche Integration der Teilergebnisse ermöglichte.

4.1 Rollenverteilung und Verantwortlichkeiten

Obwohl das Team interdisziplinär zusammenarbeitete und jedes Mitglied, wurden spezifische Verantwortungsbereiche definiert, um eine effiziente Aufgabenverwaltung zu gewährleisten:

- **Raphael Buller (Projektleitung & Architektur):** Raphael übernahm die organisatorische Leitung. Dies umfasste die Aufsetzung und Verwaltung des Repositories sowie die Initialisierung und Verwaltung des Kanban-Boards. Er transformierte die funktionalen Anforderungen in konkrete User Stories und übernahm deren Delegation. Ein wesentlicher Schwerpunkt lag zudem auf der Architekturdokumentation (Erstellung der Use-Case- und Klassendiagramme), um den Entwicklern eine fundierte Implementierungsgrundlage zu bieten. Darüber hinaus verantwortete er die Einhaltung der Deadlines, das Erstellen der schriftlichen Artefakte und die abschließende Präsentation.
- **Niklas Felix Ulbrich (Frontend & UX/UI):** Niklas war maßgeblich für die Gestaltung der Benutzeroberfläche zuständig. Sein Fokus lag auf der Implementierung des Frontends mittels Thymeleaf, Vue.js und Tailwind CSS. Dabei stellte er sicher, dass die Anforderungen an die Benutzbarkeit (Usability) und ein konsistentes Corporate Design erfüllt wurden.
- **Justus Krahl (Backend-Entwicklung):** Justus Krahl verantwortete die gesamte Backend-Architektur auf Basis von Spring Boot. Er modellierte zentrale Domänen-Entitäten und implementierte die REST-APIs für Benutzer-, Haustier-Management sowie die Marketplace-Logik. Ein Fokus lag auf Security: Er entwickelte ein JWT-basiertes Authentifizierungssystem (inkl. Session-Handling, Token-Revocation, RBAC). Er sicherte die technische Qualität durch standardisierte API-Antworten, zentrale Fehlerbehandlung und Integrationstests. Zudem automatisierte er den Entwicklungsprozess durch den Aufbau einer CI/CD-Pipeline zur automatisierten Bereitstellung und pflegte die technische Dokumentation via OpenAPI.

Trotz der fachlichen Schwerpunkte agierte das Team als **Cross-functional Team**. Jedes Mitglied unterstützte in allen Bereichen, vom Bugfixing bis zur Validierung von Logikbausteinen.

4.2 Vorgehensmodell und Methodik

Für die Entwicklung von „Pawsitters“ wurde ein inkrementelles Vorgehensmodell in Kombination mit der Kanban-Methodik gewählt. Diese Entscheidung ermöglichte eine strukturierte Komplexitätsbeherrschung und stellte sicher, dass grundlegende Systemkomponenten stabil entwickelt wurden, bevor darauf aufbauende Funktionen implementiert wurden.

4.2.1 Inkrementelle Entwicklung

Das Projekt wurde nach dem „Ground-Up“-Prinzip realisiert. Dabei stand die inkrementelle Lieferung im Vordergrund: Zuerst wurden die kritischen Infrastrukturen wie die Datenbank-Schemata und die Kern-APIs fertiggestellt. Erst nach Abschluss dieser stabilen Bausteine erfolgte die Entwicklung der darauf aufsetzenden Geschäftslogik und der Frontend-Integration.

Da im Rahmen des Projekts keine regelmäßigen Demos vor externen Stakeholdern erforderlich waren, erwies sich dieser inkrementelle Ansatz als effizienter gegenüber einem rein iterativ-evolutionären Modell. Es wurde sichergestellt, dass jede Komponente fachlich abgeschlossen war, bevor die nächste Schicht der Architektur adressiert wurde.

4.2.2 Kanban als Steuerungsinstrument

Die Wahl fiel auf Kanban, um den Fluss der Arbeitspakete (Work Items) zu visualisieren und die Zusammenarbeit im dreiköpfigen Team zu optimieren. Im Gegensatz zu starren Zeitintervallen (Sprints) lag der Fokus bei Kanban auf der kontinuierlichen Bearbeitung und Sichtbarkeit des Fortschritts. Zur Strukturierung der übergeordneten Themenbereiche wurden Epics definiert, die wiederum in detaillierte User Stories unterteilt wurden.

Das digitale Kanban-Board wurde in folgende Spalten unterteilt, um den Status jeder User Story präzise abzubilden:

- **Backlog:** Enthielt alle identifizierten User Stories, die für den Gesamtumfang des Projekts geplant, aber noch nicht zur Bearbeitung freigegeben waren.
- **Todo:** Hier befanden sich die User Stories, die für die aktuelle Arbeitsphase priorisiert wurden.
- **In Progress:** Diese Spalte unterlag einem strikten WIP-Limit (Work in Progress) von maximal drei Aufgaben gleichzeitig. Da das Team aus drei Mitgliedern bestand, wurde so sichergestellt, dass kein Multitasking stattfand und Aufgaben zügig abgeschlossen wurden, anstatt parallel mehrere Baustellen zu öffnen.
- **Test:** Nach der Implementierung wanderten Stories in diese Phase, um durch Unit-Tests oder manuelle Reviews verifiziert zu werden.
- **Finished:** Dokumentierte alle User Stories, die innerhalb der aktuellen Phase erfolgreich abgeschlossen wurden.
- **Done:** Archiv für alle User Stories aus bereits abgeschlossenen Entwicklungsphasen, um die Gesamthistorie des Projekts abzubilden.

Zur besseren Rückverfolgbarkeit (Traceability) führten wir eine verbindliche Namenslogik für Feature-Branches ein: `{Ursprungsbranch}_feature_{Featurename}` Ein konkretes Beispiel hierfür ist `m_feature_LoginButton` (wobei `m` für den Ursprung aus dem Main-Branch steht). Diese Konvention erlaubte es jedem Teammitglied, sofort den Ursprung und den Zweck eines Branches zu identifizieren.

4.3.3 Quality Gates: Review-Prozess und Rulesets

Um die Softwarequalität (gemäß ISO 25010) sicherzustellen, wurde ein striktes Ruleset in GitHub konfiguriert. Ein direkter Push auf `develop` oder `main` war technisch unterbunden.

Integrationen erfolgten ausschließlich über Pull Requests (PR). Dabei wurde das Vier-Augen-Prinzip konsequent angewendet: Jeder PR musste von mindestens einem anderen Teammitglied gesichtet, kommentiert und explizit genehmigt werden. Dieser Review-Prozess diente nicht nur der Fehlersuche, sondern auch dem internen Wissensaustausch und der Einhaltung einheitlicher Code-Standards.

4.3.4 Repository-Struktur

Das Projekt folgt einem standardisierten Aufbau für Spring-Boot-Anwendungen. Die klare Trennung in `src/main/java` (Logik), `src/main/resources` (Konfiguration, i18n, Templates) und den Test-Bereich unterstützt die Wartbarkeit des Systems. Diese Struktur ermöglichte es uns, Backend- und Frontend-Artefakte sauber zu verwalten und Kollisionen im Repository zu minimieren. Durch diese disziplinierte Nutzung von Git konnten wir sicherstellen, dass der `main`-Branch zu jedem Zeitpunkt einen auslieferbaren Stand repräsentierte und die Beiträge aller drei Mitglieder nahtlos integriert wurden.

5.4 Continuous Integration & Continuous Delivery

Um die Softwarequalität durchgängig sicherzustellen und manuelle Fehler beim Ausrollen der Anwendung zu minimieren, wurde eine vollautomatisierte CI/CD-Pipeline auf Basis von GitHub Actions implementiert. Dieser Prozess stellt sicher, dass jede Code-Änderung verifiziert wird, bevor sie in die Produktivumgebung gelangt.

Die Pipeline ist in drei logische, aufeinander aufbauende Phasen unterteilt:

1. Continuous Integration (CI): Qualitätssicherung durch Tests

Bei jedem Push oder Pull Request auf die zentralen Branches (`main` und `develop`) wird automatisch die CI-Phase gestartet.

- **Backend:** Es wird eine Java-Umgebung aufgesetzt, das Projekt mittels Maven gebaut und sämtliche Unit- sowie Integrationstests ausgeführt.
- **Frontend:** Parallel dazu wird die Node.js-Umgebung vorbereitet, die Frontend-Anwendung gebaut und automatisierte End-to-End-Tests mit **Playwright** durchgeführt.

- **Ergebnis:** Nur wenn alle Tests erfolgreich bestanden wurden, gilt der Code-Stand als stabil und die nächste Phase wird freigeschaltet.

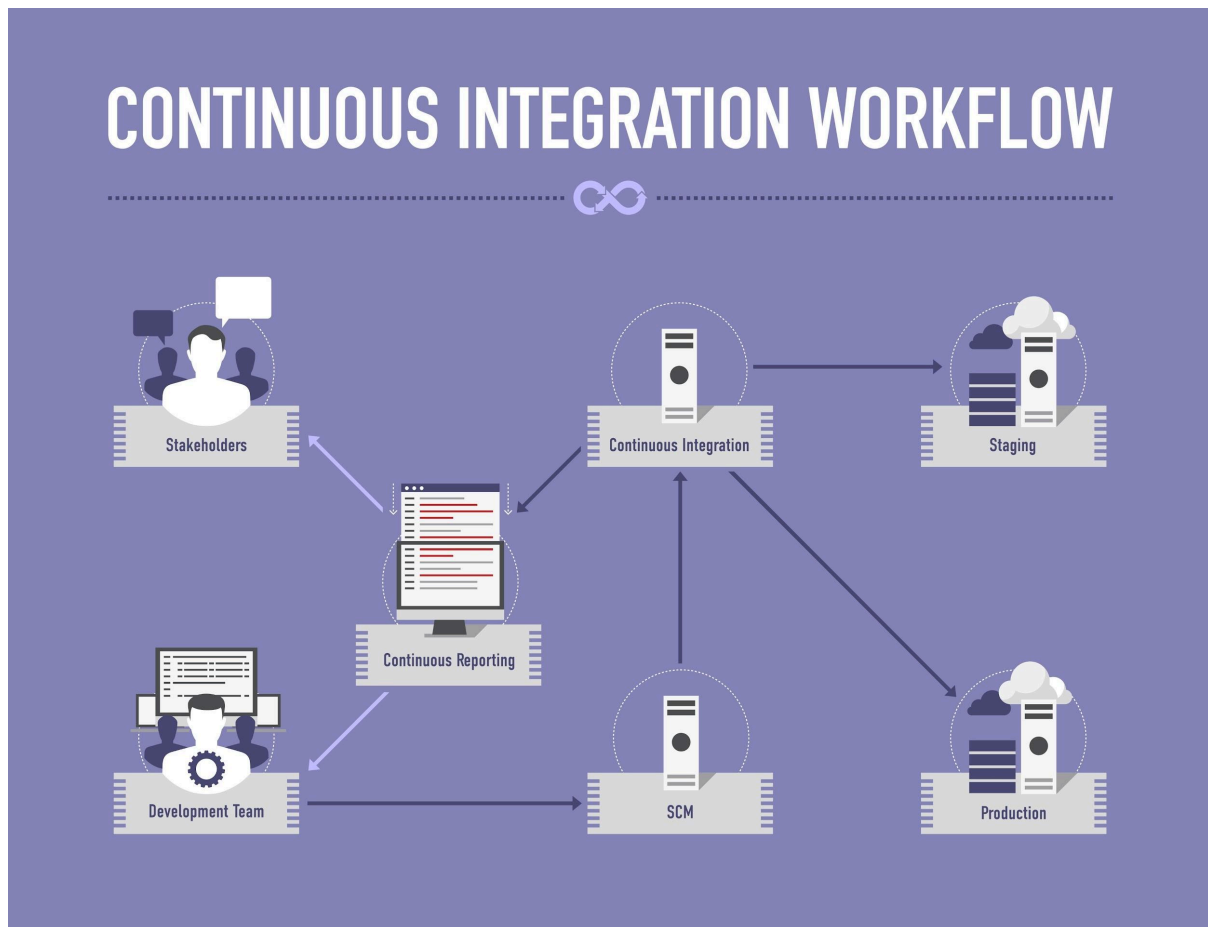


Abb. 5: Tatsächliche Visualisierung der automatisierten CI/CD-Pipeline in GitHub Actions

2. Container Build & Smoke Tests: Standardisierung und Verifikation

Sobald die Tests erfolgreich abgeschlossen sind, wird der Code in **Docker-Container** verpackt.

- **Containerisierung:** Es werden reproduzierbare Docker-Images für das Backend und das Frontend erstellt und in der GitHub Container Registry (GHCR) gespeichert.
- **Smoke Tests:** Direkt nach dem Erstellen der Images findet eine automatisierte Funktionsprüfung statt. Das System simuliert einen Start der Container und prüft über einen „Healthcheck“, ob die Anwendung korrekt hochfährt und erreichbar ist. Erst nach dieser Bestätigung wird das Image für das Deployment freigegeben.

3. Continuous Deployment (CD): Automatisierter Rollout

Nach erfolgreichen Smoke Tests auf dem `develop`-Branch erfolgt die automatische Auslieferung auf den Staging-Server (der vom Team privat gehostet wurde).

- **Sicherer Transfer:** Die Pipeline authentifiziert sich verschlüsselt auf dem Zielsystem und überträgt die notwendigen Konfigurationsdateien.
- **Aktualisierung:** Auf dem Server werden die neuesten Images automatisch heruntergeladen und die Dienste via Docker Compose neu gestartet.
- **Abschlusskontrolle:** Ein finaler automatisierter Check auf dem Live-System garantiert, dass das Deployment erfolgreich war und die Anwendung für Nutzer zur Verfügung steht.

5.4.1 Strategische Architektur-Entscheidungen im Deployment

Die Wahl der Deployment-Infrastruktur wurde gezielt auf die Projektgröße und das Team abgestimmt:

- **Einsatz von Docker:** Durch die Containerisierung wurde das Problem „In meiner Umgebung läuft es“ eliminiert. Die Anwendung läuft in der exakt gleichen Umgebung, egal ob auf dem Entwickler-Rechner oder dem Server. Dies erhöht die **Portabilität** (ISO 25010) erheblich, da der Server keine spezifischen Java- oder Node.js-Installationen benötigt.
- **Bewusste Entscheidung gegen Kubernetes:** Für den aktuellen Umfang von Pawsitters wurde auf komplexe Orchestrierungswerkzeuge wie Kubernetes verzichtet. Ein Setup mittels Docker Compose bietet für ein dreiköpfiges Team die optimale Balance zwischen geringer Komplexität und hoher Zuverlässigkeit. Diese Entscheidung spart wertvolle Ressourcen bei der Wartung und Konfiguration der Infrastruktur, ohne die Stabilität des Systems zu gefährden.

6 Einsatz von Künstlicher Intelligenz

6.1 Status Quo im Entwicklungsprozess

Im Rahmen des Pawsitters-Projekts wurde Künstliche Intelligenz (KI) gezielt als produktivitätssteigerndes Assistenzsystem in den Software Development Life Cycle (SDLC) integriert. Die architektonische Verantwortung, das Reviewing und die finalen Designentscheidungen lagen durchgehend beim menschlichen Entwicklerteam.

Konkrete Anwendungsfälle und methodischer Mehrwert

- **Software-Architektur und Modellierung:** KI-Tools wurden genutzt, um aus den vom Team erarbeiteten Spezifikationen und relationalen Schemata korrekten `Mermaid.js`-Code für die UML-Klassendiagramme und Entity-Relationship-Modelle (ERM) zu generieren. Die KI übernahm die reine Syntax-Transformation, während die fachliche Modellierungshoheit beim Team verblieb.
- **Testing und Qualitätssicherung:** Ein erheblicher Effizienzgewinn zeigte sich beim Bootstrapping von Teststrukturen. Die KI unterstützte beim Schreiben von Boilerplate-Code für die 131 JUnit-Integrationstests sowie die 34 Playwright-E2E-Tests. Zudem half sie als methodischer Prüfer, indem sie auf Basis bestehender Logik kritische Randfälle (Edge Cases) aufzeigte.
- **Code-Dokumentation und Pair Programming:** Die Wartbarkeit nach ISO 25010 wurde durch KI-gestützte Dokumentation erhöht, indem komplexe Logikblöcke standardisiert kommentiert und Markdown-Dateien (z. B. `TEST_DOCUMENTATION.md`) strukturiert ausformuliert wurden. Bei Design-Entscheidungen (z. B. der Strukturierung des Rate Limiting) diente die KI als Diskussionspartner für Best-Practice-Ansätze.

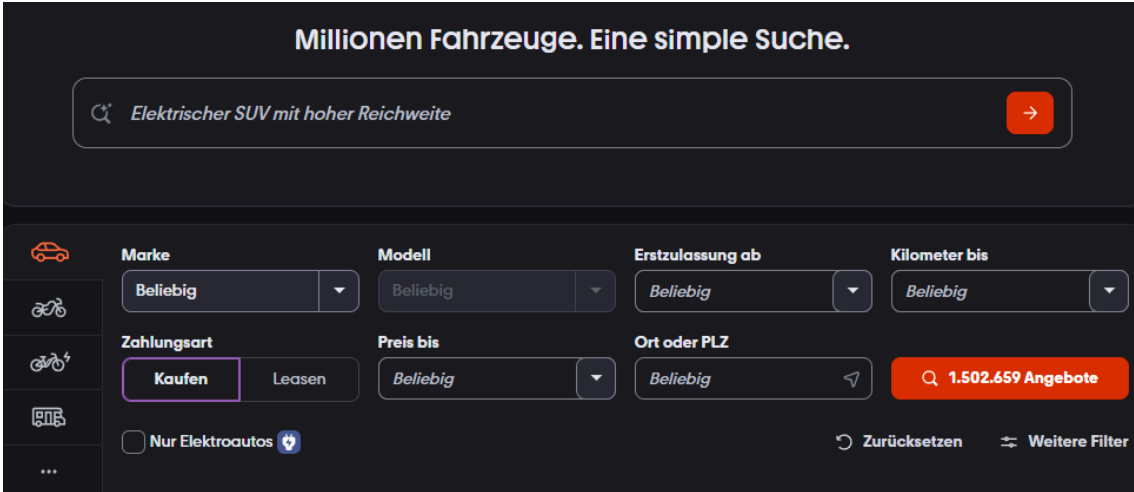
Grenzen, Risiken und Herausforderungen

- **Kontextverlust (Fehlender Systemkontext):** Bei tiefgreifenden Änderungen machten KI-Modelle isolierte Vorschläge, welche die globale Integrität des Schichten-Monolithen oder das relationale Datenmodell beschädigt hätten. Strikte manuelle Code-Reviews im Rahmen des Vier-Augen-Prinzips waren daher unerlässlich.
- **Veraltete Bibliotheken und trügerische Sicherheit:** Gelegentlich schlug die KI veraltete Spring-Boot-Abhängigkeiten oder syntaktisch inkorrekte Aufrufe vor. Besonders bei sicherheitskritischen Komponenten (wie der JWT-Validierung) war „blindes Vertrauen“ ausgeschlossen, da subtile Risiken wie CORS-Vulnerabilitäten oder unvollständige Input-Validierungen von der KI übersehen wurden.

6.2 Future Build-In: KI-gestützte Marktplatz-Suche

Um den Innovationsgrad der Plattform weiter zu steigern und KI als funktionale Systemanforderung einzuführen, ist für zukünftige Major-Releases eine **intelligente Freitext-Suche auf dem Marketplace** vorgesehen.

Aktuell erfordert die Suche nach einem passenden Gastgeber das manuelle Setzen mehrerer strukturierter Filter (Postleitzahl, Tierart, Zeitraum). Das zukünftige Feature bricht diese starre Barriere auf, indem es sich an modernen Best-Practice-Umsetzungen der Industrie orientiert, wie beispielsweise der intelligenten Freitext-Suche von Mobile.de, bei der Nutzer ihre Suchkriterien in einem einzigen, natürlichen Satz formulieren können.



The screenshot displays the Mobile.de search interface. At the top, a large search bar contains the text "Elektrischer SUV mit hoher Reichweite" with a magnifying glass icon on the left and a red arrow button on the right. Below the search bar, there are several filter sections. On the left, there are icons for car, motorcycle, scooter, and truck. The main filter area includes: "Marke" (Brand) with a dropdown menu set to "Beliebig"; "Modell" (Model) with a dropdown menu set to "Beliebig"; "Erstzulassung ab" (First registration from) with a dropdown menu set to "Beliebig"; "Kilometer bis" (Mileage up to) with a dropdown menu set to "Beliebig"; "Zahlungsart" (Payment type) with buttons for "Kaufen" (Buy) and "Leasen" (Lease); "Preis bis" (Price up to) with a dropdown menu set to "Beliebig"; "Ort oder PLZ" (Location or ZIP code) with a dropdown menu set to "Beliebig"; and a checkbox for "Nur Elektroautos" (Only electric cars). A red button on the right shows "1.502.659 Angebote" (1,502,659 offers). At the bottom right, there are links for "Zurücksetzen" (Reset) and "Weitere Filter" (More filters).

Abb. 6: Beispiel der KI-Unterstützten Suche am Beispiel von Mobile.de

Durch diese funktionale Erweiterung wird KI von einem reinen Entwicklungswerkzeug zu einem integralen Bestandteil der Anwendungsarchitektur, der die User Experience und Konversionsrate der Plattform maßgeblich optimiert.

7. Fazit

Die erfolgreiche Konzeption und Realisierung der Plattform „Pawsitters“ demonstriert die praktische Anwendbarkeit moderner Software-Engineering-Paradigmen zur Lösung eines realen Alltagsproblems. Durch den konsequenten Einsatz einer klassischen MVC-Schichtenarchitektur (Layered Monolith) konnte die im Projektportfolio geforderte funktionale Komplexität, von der feingranularen Haustier-Registrierung bis hin zur dynamischen Vermittlungslogik auf dem Marktplatz, strukturiert abgebildet und beherrscht werden. Die strategische Entscheidung gegen eine Microservice-Infrastruktur erwies sich für das dreiköpfige Entwicklerteam als ökonomisch und technisch sinnvoll, da ein unverhältnismäßiger administrativer Overhead zugunsten einer schnellen Entwicklungsgeschwindigkeit vermieden wurde.

Ein besonderer Schwerpunkt des Projekts lag auf der Einhaltung hoher Qualitätsmetriken nach. Mit einem finalen Bestand von insgesamt 165 automatisierten Tests, aufgeteilt in isolierte JUnit-Unit-Tests im Backend sowie umfassende Playwright-End-to-End-Tests im Frontend, wurde eine präzise Schutzzone gegen funktionale Regressionen etabliert, die weit

über die geforderten Mindestkriterien hinausgeht. Parallel dazu wurde durch die Umsetzung eines zustandslosen JWT-Authentifizierungsverfahrens, strikter serverseitiger Validierungen (Bean Validation) sowie dem proaktiven Ansatz des „Shift Security Left“ ein tiefgreifendes Sicherheitsbewusstsein operationalisiert. Dank der vollständigen Kapselung des Quellcodes in Docker-Container und der Orchestrierung via Docker Compose konnte das Team eine vollständige Parität zwischen der lokalen Entwicklung und dem automatisierten Deployment auf dem Staging-Server erzielen.

Für zukünftige Entwicklungszyklen (Roadmap) bietet dieses stabile Fundament hervorragende Erweiterungsmöglichkeiten. Neben den noch ausstehenden MVP-Schnitten wie der persistenten Buchungsbestätigung und der Aktivierung des In-App-Chats, stellt die geplante Integration einer KI-gestützten Freitext-Suche einen logischen Innovationsschritt dar.

Zusammenfassend zeigt das Projekt, dass der gezielte Einsatz moderner Werkzeuge wie Pipelines, agiler Kanban-Steuerung und reflektierter KI-Assistenz die Entwicklungsgeschwindigkeit drastisch erhöht, ohne dass das fundamentale Verständnis der zugrundeliegenden Softwarearchitektur verloren geht. Pawsitters präsentiert sich somit als wartbare, sichere und zukunftsfähige Softwarelösung, die mit ein paar weiteren nicht funktionalen Anforderungen sogar produktiv gehen könnte :).

Anhang

Anhang 1: Use Case Beschreibungen der Pawsitters Applikation.....	31-39
Anhang 1.1: Use Case 1 - Account erstellen.....	31
Anhang 1.2: Use Case 2 - Konto verwalten.....	32
Anhang 1.3: Use Case 3 - Gastgeberprofil anlegen und verwalten.....	33
Anhang 1.4: Use Case 4 - Haustier registrieren.....	34
Anhang 1.5: Use Case 5 - Buchungsanfrage annehmen/ablehnen.....	35
Anhang 1.6: Use Case 6 - Suche & Marketplace.....	36
Anhang 1.7: Use Case 7 - Angebot erstellen.....	37
Anhang 1.8: Use Case 8 - Angebot verwalten.....	38
Anhang 1.9: Use Case 9 - Nachrichten senden (In-App Chat).....	39
 Anhang 2: API Schnittstellen der Paw Sitters Applikation.....	 40-41

Anhang 1: Use Case Beschreibung der Pawsitters Applikation

Anhang 1.1: Use Case 1 - Account erstellen

Feld	Beschreibung
Name	UC1 – Account erstellen
Beschreibung	Ein neuer Nutzer registriert sich auf der Pawsitters-Plattform, indem er seine persönlichen Daten (Name, E-Mail, Passwort, Telefonnummer) eingibt und ein Konto anlegt.
Akteure	Benutzer (unregistriert)
Auslöser	Der Nutzer möchte die Plattform nutzen und ruft die Registrierungsseite auf.
Vorbedingung	Es existiert noch kein Konto mit der angegebenen E-Mail-Adresse.
Standardablauf	1. Nutzer ruft die Registrierungsseite auf. 2. Nutzer füllt das Registrierungsformular aus (Vorname, Nachname, E-Mail, Passwort, Telefonnummer). 3. System validiert die Eingaben (Format, Pflichtfelder, Passwortstärke). 4. System prüft, ob die E-Mail-Adresse bereits vergeben ist. 5. System speichert das Konto mit gehashtem Passwort (BCrypt). 6. Nutzer wird zur Startseite/Dashboard weitergeleitet.
Alternative Pfade	A1: E-Mail bereits registriert: Das System erkennt in Schritt 4, dass die E-Mail bereits existiert. Der Nutzer erhält eine Fehlermeldung und wird auf die Login-Seite verwiesen. A2: Ungültige Eingabe: Die Validierung in Schritt 3 schlägt fehl (z.B. falsches E-Mail-Format, Passwort zu schwach, Pflichtfeld fehlt). Der Nutzer erhält eine spezifische Fehlermeldung und muss die Eingaben korrigieren.
Nachbedingung	Ein neuer User-Datensatz existiert in der Datenbank. Der Nutzer ist erfolgreich eingeloggt.

Anhang 1.2: Use Case 2 - Konto verwalten

Feld	Beschreibung
Name	UC2 – Konto verwalten
Beschreibung	Ein angemeldeter Benutzer kann seine Profildaten (Name, E-Mail, Telefonnummer, Passwort) einsehen und aktualisieren sowie sein Konto löschen.
Akteure	Benutzer (eingeloggt)
Auslöser	Der Benutzer navigiert zu seinem Profilbereich und möchte Daten ändern.
Vorbedingung	Der Benutzer ist auf der Plattform angemeldet (authentifiziert).
Standardablauf	1. Benutzer ruft die Profilseite auf. 2. System zeigt die aktuellen Kontodaten an. 3. Benutzer ändert die gewünschten Felder (z. B. Telefonnummer, Passwort). 4. System validiert die geänderten Eingaben. 5. System speichert die aktualisierten Daten. 6. Benutzer erhält eine Bestätigung der erfolgreichen Aktualisierung.
Alternative Pfade	A1: Ungültige Eingabe: Die Validierung in Schritt 4 schlägt fehl (z.B. falsches E-Mail-Format, Passwort zu schwach). Der Benutzer erhält eine spezifische Fehlermeldung und muss die Eingaben korrigieren. A2: Passwortänderung: Wenn das Passwort geändert wird, muss das System das alte Passwort zur Verifikation abfragen. Schlägt die Verifikation fehl, wird die Änderung abgelehnt. A3: Konto löschen: Benutzer wählt die Option "Konto löschen". Das System fordert eine Bestätigung und löscht nach erfolgreicher Bestätigung (z.B. durch erneute Passworteingabe) alle zugehörigen User-Datensätze und loggt den Nutzer aus.
Nachbedingung	Die User-Daten in der Datenbank sind aktualisiert oder der User-Datensatz ist gelöscht.

Anhang 1.3: Use Case 3 - Gastgeberprofil anlegen und verwalten

Feld	Beschreibung
Name	UC3 – Gastgeberprofil anlegen und verwalten
Beschreibung	Ein registrierter Benutzer kann ein dediziertes Profil als Gastgeber erstellen, seine Betreuungsleistungen (Art der Betreuung, Preise, Verfügbarkeiten) definieren und bei Bedarf aktualisieren.
Akteure	Benutzer (eingeloggt), System
Auslöser	Der Benutzer möchte seine Dienste als Tiersitter auf der Plattform anbieten.
Vorbedingung	Der Benutzer ist angemeldet und hat noch kein aktives Gastgeberprofil.
Standardablauf	1. Benutzer navigiert zum Bereich "Gastgeber werden". 2. Benutzer füllt das Formular für das Gastgeberprofil aus (Leistungen, Preise, verfügbare Zeiträume, Beschreibung der Unterkunft). 3. System validiert die Eingaben (Pflichtfelder, Preisformat). 4. System speichert das neue HostProfile und verknüpft es mit dem User-Datensatz. 5. Das Profil wird auf dem Marktplatz freigeschaltet.
Alternative Pfade	A1: Profil aktualisieren: Der Benutzer ruft das bestehende Gastgeberprofil auf und ändert einzelne Felder (z.B. neue Preise, neue Verfügbarkeiten). Das System speichert die Änderungen und aktualisiert das Profil auf dem Marktplatz. A2: Ungültige Eingabe: Die Validierung in Schritt 3 schlägt fehl. Der Benutzer erhält eine Fehlermeldung und muss die Eingaben korrigieren.
Nachbedingung	Ein aktiver HostProfile-Datensatz existiert, der Benutzer ist nun als Gastgeber auf dem Marktplatz sichtbar und suchbar.

Anhang 1.4: Use Case 4 - Haustier registrieren

Feld	Beschreibung
Name	UC4 – Haustier registrieren
Beschreibung	Ein Tierhalter legt ein neues Haustier in seinem Profil an und gibt relevante Informationen wie Name, Spezies, Rasse, Alter und besondere Bedürfnisse ein.
Akteure	Tierhalter (eingeloggt)
Auslöser	Der Tierhalter möchte ein neues Haustier auf der Plattform erfassen.
Vorbedingung	Der Benutzer ist angemeldet und hat die Rolle „Tierhalter“.
Standardablauf	1. Tierhalter navigiert zum Bereich „Meine Haustiere“. 2. Tierhalter klickt auf „Haustier hinzufügen“. 3. Tierhalter füllt das Formular aus (Name, Spezies, Rasse, Alter, besondere Bedürfnisse). 4. System validiert die Pflichtfelder. 5. System speichert das Haustier und verknüpft es mit dem Besitzerkonto. 6. Das neue Haustier erscheint in der Übersicht des Tierhalters.
Alternative Pfade	A1: Ungültige Eingabe: Die Validierung in Schritt 4 schlägt fehl (z.B. fehlendes Pflichtfeld 'Name'). Der Tierhalter erhält eine spezifische Fehlermeldung und muss die Eingaben korrigieren.
Nachbedingung	Ein neuer Pet-Datensatz ist in der Datenbank vorhanden und mit dem User-Datensatz des Tierhalters verknüpft.

Anhang 1.5: Use Case 5 – Buchungsanfrage annehmen/ablehnen

Feld	Beschreibung
Name	UC5 – Buchungsanfrage annehmen/ablehnen
Beschreibung	Ein Gastgeber empfängt eine offene Buchungsanfrage von einem Tierhalter, prüft die Details und entscheidet, diese anzunehmen oder abzulehnen.
Akteure	Gastgeber (eingeloggt), System
Auslöser	Der Gastgeber erhält eine Benachrichtigung über eine neue Buchungsanfrage.
Vorbedingung	Der Gastgeber ist angemeldet; Es existiert ein BookingRequest-Datensatz mit dem Status 'Offen', der dem Gastgeber zugeordnet ist.
Standardablauf	1. Gastgeber navigiert zur Übersicht der offenen Anfragen. 2. Gastgeber wählt eine offene Anfrage aus, um die Details (Zeitraum, Tier, Tierhalter) einzusehen. 3. Gastgeber klickt auf „Annehmen“ oder „Ablehnen“. 4a. Bei Annahme: System ändert den Status des BookingRequest-Datensatzes auf 'Bestätigt'. System prüft/blockiert die Verfügbarkeit des Gastgebers für den Zeitraum. Tierhalter erhält eine Benachrichtigung über die Bestätigung. 4b. Bei Ablehnung: System ändert den Status des BookingRequest-Datensatzes auf 'Abgelehnt'. Tierhalter erhält eine Benachrichtigung über die Ablehnung.
Alternative Pfade	A1: Konflikt bei Annahme: Das System erkennt bei Schritt 4a, dass die Buchung mit einer anderen, parallel abgewickelten Buchung kollidiert. Die Annahme wird verweigert, der Gastgeber erhält eine Fehlermeldung.
Nachbedingung	Der Status des BookingRequest-Datensatzes ist entweder 'Bestätigt' oder 'Abgelehnt'. Die Verfügbarkeit des Gastgebers wurde entsprechend angepasst.

Anhang 1.6: Use Case 6 – Suche & Marketplace

Feld	Beschreibung
Name	UC6 – Suche & Marketplace
Beschreibung	Ein Tierhalter kann auf dem Marketplace nach verfügbaren Gastgebern oder Betreuungsangeboten suchen und diese filtern (z. B. nach Zeitraum, Spezies, Ort, Preis).
Akteure	Tierhalter (eingeloggt)
Auslöser	Der Tierhalter sucht eine Betreuung für sein Haustier und möchte passende Angebote finden.
Vorbedingung	Der Tierhalter ist angemeldet; Es existieren aktive Gastgeberprofile mit definierten Verfügbarkeiten auf der Plattform.
Standardablauf	1. Tierhalter navigiert zum Marketplace. 2. Tierhalter gibt Suchkriterien ein (z. B. Ort, Zeitraum, Tierart, Preisrahmen). 3. System sendet die Suchanfrage an den Backend-Service. 4. Der Service filtert die HostProfile basierend auf den Kriterien und der Verfügbarkeit. 5. System zeigt die gefilterten und sortierten Angebote der Gastgeber in einer Ergebnisliste an. 6. Tierhalter wählt ein Angebot aus, um das detaillierte Profil einzusehen (UC4).
Alternative Pfade	A1: Keine Ergebnisse: Das System findet keine passenden Angebote für die gewählten Kriterien. Der Tierhalter erhält eine entsprechende Meldung und kann die Suchkriterien anpassen. A2: Filter ändern: Der Tierhalter passt die Suchkriterien in Schritt 2 an, um die Ergebnisse zu verfeinern oder zu erweitern. Das System wiederholt die Schritte 3-5 mit den neuen Kriterien.
Nachbedingung	Die Ergebnisliste des Marketplace ist auf dem Bildschirm des Tierhalters sichtbar; optional wurde ein Host-Profil aufgerufen.

Anhang 1.7: Use Case 7 – Angebot erstellen

Feld	Beschreibung
Name	UC7 – Angebot erstellen
Beschreibung	Ein Gastgeber erstellt eigenständig ein Betreuungsangebot und veröffentlicht es auf dem Marketplace. Das Angebot enthält den Betreuungszeitraum, den Preis pro Tag, die angebotenen Leistungen sowie die akzeptierten Tierarten.
Akteure	Gastgeber (eingeloggt)
Auslöser	Der Gastgeber möchte seine Betreuungskapazitäten anbieten und erstellt ein neues Angebot.
Vorbedingung	Der Gastgeber ist angemeldet und hat die Rolle „Gastgeber“.
Standardablauf	1. Gastgeber navigiert zu „Angebot erstellen“. 2. Gastgeber füllt das Angebotsformular aus: Betreuungszeitraum (Von–Bis), Preis pro Tag, angebotene Leistungen (z. B. Fütterung, Spaziergänge, Übernachtung) und akzeptierte Tierarten (z. B. Hund, Katze, Vogel). 3. System validiert die Pflichtfelder (Zeitraum, Preis > 0, mind. eine Tierart). 4. System speichert das Angebot und veröffentlicht es auf the Marketplace.
Alternative Pfade	A1: Ungültige Eingabe: Die Validierung in Schritt 3 schlägt fehl (z.B. Preis 0 oder negatives Datum). Der Gastgeber erhält eine Fehlermeldung und muss die Eingaben korrigieren. A2: Angebot aktualisieren: Der Gastgeber wählt ein bestehendes Angebot aus und ändert die Details (Preis, Leistungen, Zeitraum). Das System speichert die Änderungen und aktualisiert die Anzeige auf dem Marketplace.
Nachbedingung	Ein neuer/aktualisierter Offer-Datensatz existiert und ist aktiv auf dem Marketplace sichtbar und suchbar.

Anhang 1.8: Use Case 8 – Angebot verwalten

Feld	Beschreibung
Name	UC8 – Angebot verwalten
Beschreibung	Ein Gastgeber kann seine erstellten Angebote einsehen, bearbeiten (solange noch ausstehend) oder zurückziehen.
Akteure	Gastgeber (eingeloggt)
Auslöser	Der Gastgeber möchte den Status oder Inhalt seiner Angebote überprüfen oder ändern.
Vorbedingung	Der Gastgeber ist angemeldet und hat mindestens ein Angebot erstellt.
Standardablauf	1. Gastgeber öffnet die Übersicht „Meine Angebote“. 2. System zeigt alle Angebote des Gastgebers mit Status (PENDING, ACCEPTED, REJECTED) an. 3. Gastgeber wählt ein Angebot aus. 4a. Bei ausstehenden Angeboten: Gastgeber kann Preis oder Nachricht anpassen und speichern. 4b. Gastgeber kann das Angebot zurückziehen. 5. System aktualisiert den Angebotsstatus und informiert ggf. den Tierhalter.
Alternative Pfade	A1: Angebot bereits bestätigt/abgelehnt: In Schritt 4 wird dem Gastgeber angezeigt, dass das Angebot nicht mehr bearbeitet werden kann, da der Status bereits final ist.
Nachbedingung	Die Angebotsdaten sind aktualisiert oder der Angebotsstatus ist auf 'Zurückgezogen' gesetzt.

Anhang 1.9: Use Case 9 – Nachrichten senden (In-App Chat)

Feld	Beschreibung
Name	UC9 – Nachrichten senden (In-App Chat)
Beschreibung	Ein eingeloggter Benutzer (Tierhalter oder Gastgeber) kann über das integrierte Chat-System eine direkte Nachricht an seinen Kommunikationspartner (entweder ein potenzieller Gastgeber oder ein Tierhalter) senden, um Details zur Betreuung zu klären.
Akteure	Tierhalter (eingeloggt), Gastgeber (eingeloggt), System
Auslöser	Der Tierhalter oder der Gastgeber möchte nach der ersten Kontaktaufnahme (z.B. nach einer Buchungsanfrage) weitere Details klären oder Fragen beantworten.
Vorbedingung	Beide Akteure sind angemeldet; Es existiert mindestens eine aktive oder vergangene Interaktion (z.B. eine Buchungsanfrage) zwischen den beiden Benutzern, die den Chat-Raum öffnet.
Standardablauf	1. Benutzer navigiert zur Chat-Übersicht. 2. Benutzer wählt den Chat mit dem gewünschten Gesprächspartner aus. 3. Benutzer gibt eine Nachricht ein und klickt auf „Senden“. 4. System speichert die Nachricht im Chat-Verlauf und versieht sie mit einem Zeitstempel. 5. Der Empfänger erhält eine Echtzeit-Benachrichtigung über die neue Nachricht. 6. Empfänger kann auf die Nachricht antworten (Schritte 3-5).
Alternative Pfade	A1: Chat nicht verfügbar: Wenn keine vorherige Interaktion (Anfrage/Angebot) existiert, ist der direkte Chat möglicherweise nicht freigeschaltet. Der Benutzer erhält eine Meldung, dass zuerst eine Anfrage gestellt werden muss. A2: Überlange Nachricht: Die eingegebene Nachricht überschreitet die maximale Zeichenanzahl. Der Benutzer erhält eine Fehlermeldung und muss die Nachricht kürzen.
Nachbedingung	Die gesendete Nachricht ist persistent in der Datenbank gespeichert und für beide Chat-Teilnehmer im Interface sichtbar.

Anhang 2: API Schnittstellen der Paw Sitters Applikation

HTTP-Methode	Endpunkt	Beschreibung	Bereich (Tag)
POST	/api/auth/register	Registrieren und einloggen	Auth
POST	/api/auth/login	Einloggen	Auth
POST	/api/auth/logout	Ausloggen	Auth
GET	/api/auth/session	Sessionstatus abrufen	Auth
POST	/api/users/register	Benutzer registrieren	Users
GET	/api/users/{id}	Benutzer abrufen	Users
PUT	/api/users/{id}	Benutzer vollständig aktualisieren	Users
PATCH	/api/users/{id}	Benutzer partiell aktualisieren	Users
DELETE	/api/users/{id}	Benutzer löschen	Users
GET	/api/users/me	Aktuellen Benutzer abrufen	Users
GET	/api/users/mailExists	E-Mail prüfen	Users
PATCH	/api/users/{id}/roles	Benutzerrolle ändern	Users
POST	/api/users/{id}/profile-image	Profilbild hochladen	Users
DELETE	/api/users/{id}/profile-image	Eigenes Profilbild löschen	Users

GET	<code>/api/pets</code>	Eigene Haustiere abrufen	Pets
POST	<code>/api/pets</code>	Haustier erstellen	Pets
PUT	<code>/api/pets/{id}</code>	Haustier aktualisieren	Pets
DELETE	<code>/api/pets/{id}</code>	Haustier löschen	Pets
POST	<code>/api/pets/{id}/image</code>	Haustierbild hochladen	Pets
POST	<code>/api/offers</code>	Betreuungspaket als Entwurf erstellen	Offers
GET	<code>/api/offers/{id}</code>	Eigenes Betreuungspaket abrufen	Offers
PATCH	<code>/api/offers/{id}/publish</code>	Betreuungspaket veröffentlichen	Offers
PATCH	<code>/api/offers/{id}/withdraw</code>	Betreuungspaket zurückziehen	Offers
GET	<code>/api/marketplace/hosts</code>	Gastgeber abrufen	Marketplace
GET	<code>/api/marketplace/offers</code>	Veröffentlichte Betreuungspakete abrufen	Marketplace
GET	<code>/api/marketplace/hosts/search</code>	Gastgeber nach Tierart und Postleitzahl suchen	Marketplace
GET	<code>/api/marketplace/filters</code>	Verfügbare Marketplace-Filter abrufen	Marketplace